AI-NATIVE VERSION CONTROL SYSTEM

Vault-AI

Complete Feature List — All Phases

Every capability built into Vault-AI from Phase 1 (Core VCS) through Phase 9 (Perfect 100). Each feature is documented with its module, description, and implementation details.

18	39	191
CORE MODULES	DISTINCT FEATURES	IMPLEMENTATION POINTS

22	9
CLI COMMANDS	PHASES

1 Core Version Control Engine

Module: store.py + utils.py

🏗️ vault init — Repository Initialization NEW

Creates the `.vault/` hidden directory with full object store structure. Sets up `HEAD`, `refs/heads/main`, and all sub-directories for objects, metadata, and config. Unlike Git, zero configuration required — works immediately.

- No configuration wizard required — single command
- Automatically creates .vault/objects/, .vault/refs/, .vault/metadata/
- Sets HEAD → refs/heads/main (branch pointer)
- Safe to re-run in an existing repo (no data loss)

💾 vault save — Direct Commit (No Staging Area) Core

Commits the entire working tree atomically. Eliminates `git add` + `git commit` — one command handles everything. AI auto-generates commit messages if none provided.

- Zero staging area — commits everything in one shot
- AI auto-generates commit messages from the diff if -m is omitted
- Full Merkle tree snapshot of all project files on every commit
- Cycle-Time telemetry — prints snapshot speed in milliseconds
- --force flag bypasses all security guards for emergency commits

🌳 Merkle Tree Object Store — Tamper-Proof Storage Core

Every file is SHA-256 hashed into a content-addressed object store under `.vault/objects/`. Commit objects reference tree objects which reference blob objects, forming an immutable cryptographic chain.

- SHA-256 content-addressed storage for all files
- Tree → Blob object hierarchy mirroring the directory structure
- Deduplication: identical files stored only once across all commits
- zlib compression on every object to minimize disk footprint
- Immutable: altering any historical byte breaks the entire chain

⏪ vault undo — Infinite Time-Travel Revert

Core

Reverts the repository to the parent of the current HEAD, restoring the working tree files to their prior state. Supports sparse (lazy) mode.

- vault undo last — restores working tree from parent commit
- Tracked file restoration: only files that changed are rewritten
- --sparse flag for lazy undo without full disk extraction
- Updates the branch HEAD pointer correctly after undo

📜 vault history — Commit Log

Core

Displays the full commit log with timestamps and messages. Configurable limit.

- -n / --limit flag controls how many entries to display (default 20)
- Displays commit SHA (10-char prefix), timestamp, and message
- Walks the parent chain from HEAD backwards

🌿 vault create branch — Branch Management

Core

Creates a new named branch at the current HEAD position.

- vault create branch — intent-based natural language command
- Updates refs/heads/ to point at the current commit
- Does not switch branches — stays on current branch

🗺️ vault map — Visual ASCII Branch Map

Visual

Renders a visual graph of the commit history in the terminal, showing branch divergence, merge points, and chronological progression.

- -n / --limit flag to cap number of displayed commits
- Color-coded branch labels
- Shows commit SHA, timestamp, message, and branch tags

2 Semantic Diff Engine

Module: diff_engine.py

🔍 vault diff — Semantic File Diff

Smart

Computes line-level diffs between the working tree and the last committed snapshot. Works without Git, using the internal object store.

- Compares working tree files against HEAD blob objects
- Shows files added, modified, and deleted
- Line-level + / - annotation with file name headers
- -v / --verbose flag for unified full-context diff output
- Used internally by Secret Guard, Anomaly Detection, and AI commit message generation

3 Environment Snapshotting

Module: env_snapshot.py

🌍 vault env show — Environment Snapshot Viewer

Reproducibility

Every commit captures a full environment snapshot: Python version, OS, hostname, installed packages, and key environment variables. `vault env show` displays this data.

- Captures Python version, OS platform, machine architecture
- Auto-reads requirements.txt and package.json for package lists
- Stores key env vars: PATH, SHELL, LANG, TERM
- Indexed per-commit SHA for historical retrieval
- vault env show — view snapshot from any past commit

🔄 vault env diff — Environment Change Detector

Reproducibility

Compares the current runtime environment against any historical snapshot, showing packages added, removed, or version-changed.

- Detects newly installed packages vs the stored snapshot
- Detects removed packages since that commit
- Detects version-changed packages (e.g. requests 2.28 → 2.31)
- vault env diff — diff against a specific historical commit
- Enables perfect environment reproducibility for data science teams

4 Secret Guard — Credential Protection

Module: secret_guard.py

Pre-Commit Secret Scanner

Security

Automatically scans every file before committing. Detects hardcoded credentials, tokens, private keys, and high-entropy strings using both regex patterns and entropy analysis.

- Detects AWS Access Key IDs and Secret Access Keys
- Detects Google Cloud API keys and service account credentials
- Detects GitHub tokens (ghp_, gho_, ghx_ prefixes)
- Detects generic API_KEY / TOKEN / PASSWORD = 'value' patterns
- Detects RSA/PEM private key blocks (-----BEGIN PRIVATE KEY-----)
- High-entropy string detection (Shannon entropy > threshold)
- Blocks the commit and prints file + line number + match
- Auto-quarantines secrets to .vault_private log file

Secret Allowlist (.vault-secrets-allow)

Security

Fine-grained control over which files to exempt from scanning.

- Add a filename to .vault-secrets-allow to whitelist the entire file
- Add # vault-ok comment inline to suppress a single line
- vault save --force bypasses all checking for emergency commits

5 AI Brain — Language Model Switchboard

Module: llm.py + config.py

🤖 Ollama (Default) — Local Zero-Cost AI

Default

Ollama is the hardcoded default. If no config exists, the system uses `localhost:11434` without any API key. Supports any Ollama-compatible model.

- Default: localhost:11434 — no configuration required
- OLLAMA_URL and OLLAMA_MODEL env vars for override
- Sends format: json instruction for structured outputs
- Completely free — runs on your own hardware

💎 Gemini Adapter — Google Free Tier Cloud

Optional

Plug-and-play adapter for Google Gemini. Dormant until GEMINI_API_KEY is set.

- Uses Gemini REST API directly — zero Python SDK dependencies
- Default model: gemini-1.5-flash (free tier)
- GEMINI_MODEL env var to override model
- vault config use gemini — activates this adapter

🧠 OpenAI Adapter — GPT Models

Optional

Plug-and-play adapter for OpenAI GPT models. Dormant until OPENAI_API_KEY is set.

- Uses OpenAI chat completions API (v1/chat/completions)
- Default model: gpt-3.5-turbo; OPENAI_MODEL to override
- vault config use openai — activates this adapter

🔧 Custom Script Adapter — Bring Your Own AI

Advanced

Link any personal .py or .sh script as the AI brain. The script receives the prompt on stdin and returns output on stdout.

- vault config use-custom /path/to/script.py
- Script must be executable (chmod +x)
- Receives prompt via stdin; writes response to stdout
- VAULT_REQUIRE_JSON=1 injected when structured output needed
- Enables integration with any private or on-premise LLM

vault config — AI Switchboard Management

Management

Full CLI management of the active AI engine stored in .vault/config.json.

- vault config list-ai — shows all adapters with active indicator
- vault config use — switch engine
- vault config use-custom — register custom script
- vault config reset — instantly reverts brain to Ollama default

JSON Format Enforcement

Schema

When the VCS core requires structured output (merge decisions, anomaly analysis, etc.), all adapters receive automatic JSON schema injection into the prompt.

- Prepends explicit JSON-only instructions to every structured prompt
- Strips markdown code fences from LLM responses before parsing
- Protects downstream JSON parsers from format drift

6 Intelligence Layer — AI-Powered Features

Module: ai_brain.py + story.py + self_heal.py + ghost_pair.py

✍️ AI Commit Message Generator AI

When you run `vault save` without `-m`, AI analyzes the diff and writes a descriptive commit message. You can accept, reject, or edit it.

- Reads the unified diff of all changed files
- Sends diff to active LLM with a summarization prompt
- Interactive: Accept [Y/n/edit] prompt before finalizing
- Falls back to manual entry if AI is unavailable

📖 vault story — Development Story Mode AI

Generates a human-readable Markdown narrative from recent commit history, readable by non-technical stakeholders.

- --days N flag sets the time window (default: 7 days)
- Lists commit count, date range, and full chronological log
- --pdf flag exports the story as a PDF document
- Useful for sprint retrospectives, client updates, or changelogs

🔧 Self-Healing Commits — AI Lint Auto-Fix AI

Before every commit, runs linters (flake8 for Python, eslint for JS). If errors are found, sends them to Ollama for an AI-generated fix proposal.

- Runs flake8 / pylint on Python files automatically
- Runs eslint on JavaScript files if available
- Sends errors + code context to LLM for fix suggestions
- Displays fix proposal for user acceptance before applying
- Blocks commit on unfixed errors (use --force to skip)

👥 Ghost Pair Detection — AST Similarity Engine AI

Scans the working tree for duplicate or near-duplicate functions using AST token comparison. Warns about redundant implementations.

- Parses Python AST to extract function signatures and bodies
- Computes token-level similarity score between function pairs

- Warns if similarity > 80% (configurable threshold)
- Suggests which function is likely the redundant copy
- Non-blocking — shown as a warning, not a hard stop

7 Semantic Search

Module: search.py

🔎 vault search — Natural Language Commit Search RAG

Search through commit history and code using natural language queries.

- Primary mode: ChromaDB vector embeddings for semantic similarity search
- Fallback mode: Substring search across commit messages if ChromaDB absent
- vault search "authentication bug" — finds conceptually related commits
- Returns matching commit SHAs, timestamps, and relevant snippets
- Gracefully degrades without requiring pip install

8 Smart Reminders & Notifications

Module: reminders.py

⚠️ Large Change Whisper — 500+ Line Warning UX

Automatically detects when a single commit contains more than 500 lines of changes and nudges the user to consider splitting the work.

- Counts total added + removed lines across all changed files
- Triggers a soft warning above 500 lines: does NOT block the commit
- Encourages smaller, more reviewable commits
- Helpful for code review workflows and PR hygiene

9 High-Scale Versioning — Shadow Storage & Lazy Load

Module: store.py + lazy.py

Shadow Storage — Large File Threshold Tracker

Performance

Files exceeding 50MB are automatically offloaded from the Merkle tree to a separate Shadow Storage location, keeping the core object store fast and lean.

- Threshold: 50MB — configurable in store.py
- Large file → SHA-256 hashed and moved to .vault/objects/large/
- Original filepath replaced with a .vlink JSON metadata file
- .vlink contains: { "vlink": true, "sha": "...", "size": 54000000 }
- On vault undo / restore: .vlink automatically swapped back to real file
- Supports AI model weights, datasets, video files, compiled binaries
- Makes commits containing large files blazing fast

vault hydrate — Lazy-Load / Sparse Indexing Engine

Performance

For monorepo efficiency, Vault-AI supports sparse checkout. Only extract files from the vault when you actually need to work on them.

- vault hydrate . — extracts all sparse manifest files to disk
- vault hydrate src/models/ — extracts only a specific subdirectory
- vault hydrate model.bin — extracts a single specific file
- Reads .vault/sparse_manifest.json to know what needs hydration
- Essential for large monorepos where full checkout is wasteful

10 AI Judge — Intelligent Merge System

Module: merge.py + conflict.py

⚖️ vault merge — AI-Powered Branch Merge

AI High-Trust

Instead of producing raw merge conflict markers, Vault-AI's AI Judge analyzes the intent of both branches and resolves conflicts semantically.

- Reads the commit messages from both branches as 'intent' signals
- Sends the conflicting file contents + intent to the LLM for judgment
- If confidence ≥ 90%: auto-resolves and commits the merge immediately
- If confidence < 90%: PAUSES and shows the reasoning summary
- High-Trust prompt: 'I merged using [X] logic but I am only 82% sure. Accept? [y/N]'
- Non-conflicting files from the source branch are merged automatically
- Falls back to standard diff-marker format if AI is unavailable

🔮 vault predict — Predictive Conflict Resolution

AI

Runs a conflict simulation BEFORE you merge, using AI to identify which files will conflict and why.

- vault predict — analyze any two branches
- Detects files modified in both branches simultaneously
- AI analyzes logic dependencies between modified symbols
- Warns about API contract changes that could break consumers
- Zero writes to disk — purely analytical, non-destructive

11 Security Suite — Hacker Defense

Module: integrity.py + anomaly.py + dep_audit.py

🔗 Tamper-Proof History — Merkle Chain Verifier

Security

On every vault scan, the entire object graph is re-validated from HEAD to genesis. Any mutation triggers an immediate LOCKDOWN.

- Re-hashes every object in the chain and compares against stored SHAs
- Uses zlib.decompressobj() to detect trailing garbage bytes in compressed objects
- Detects byte-level tampering that standard checksum comparisons miss
- On failure: writes .vault/LOCKDOWN — blocks all further commits
- Prints lockdown banner: 🔐 LOCKDOWN ENGAGED: Repository is locked.
- Protects against insider threats, compromised CI/CD pipelines, and disk-level attacks

🚨 AI Anomaly Detection — Suspicious Logic Scanner

Security

Every commit is scanned for 8 categories of dangerous code patterns using both static regex and optional Ollama AI confirmation.

- eval() / exec() — dynamic code execution
- socket.connect / urllib.request — hidden network calls
- os.chmod / subprocess chmod — permission escalation
- base64.b64decode + __import__ — obfuscated imports
- Hardcoded passwords, tokens, API keys in code strings
- /dev/tcp, nc -e, bash -i >& — reverse shell patterns
- ctypes.CDLL / ctypes.windll — native DLL access
- shutil.rmtree('/') — destructive root-level deletion
- Ollama AI optionally confirms or dismisses static heuristic findings
- Blocks commit; use --force to override

🛡️ Dependency Audit — OSV.dev CVE Scanner

Security

On every commit or vault scan, checks all Python and JavaScript dependencies against the Open Source Vulnerability database (osv.dev).

- Parses requirements.txt for Python packages
- Parses package.json for Node.js packages
- Sends package names + versions to https://api.osv.dev/v1/query
- Flags packages with known CVEs and shows vulnerability IDs

- Non-blocking warning by default — does not stop the commit

12 Time Machine — Snapshots & File Watcher

Module: snapshot.py + watcher.py

📸 vault snapshot — Manual Time-Machine Checkpoint

Safety

Creates a labeled checkpoint separate from the main commit log. Useful for 'before big refactor' saves.

- vault snapshot -l 'before-big-refactor' — custom label
- Stored as a standard commit with snapshot:: prefix in the message
- Can be restored via vault undo
- Default label: 'manual' if no -l provided

👁️ vault watch — Background Autosave Daemon

Safety

Starts a background polling daemon that auto-commits the project at regular intervals.

- vault watch — default: saves every 30 seconds
- vault watch -i 60 — configurable interval in seconds
- Creates commits with message 'auto-save [timestamp]'
- Continues until Ctrl+C
- Essential safety net for sessions without natural commit points

13 Virtual Sandbox

Module: sandbox.py

🧪 vault sandbox — Isolated Experiment Environment

Safety

Creates an isolated copy of the current working state for risky experiments. Changes inside the sandbox never affect your real commit history unless you explicitly merge them.

- vault sandbox — enters isolated working environment
- vault sandbox status — shows if you're inside a sandbox
- vault sandbox exit — discards all sandbox changes cleanly
- vault sandbox merge — promotes sandbox changes back to working tree
- Prevents accidental commits during exploratory refactors
- Internally saves a '.sandbox' snapshot before entering

14 Policy-as-Code — Guardrail Engine

Module: policy.py

📜 .vault_policy — Code Quality Policy Enforcement

Governance Phase 9

Define project-wide rules in a .vault_policy JSON file. The engine evaluates every commit against these rules and blocks violations.

- no_console_logs: true — blocks print() / console.log() in source files
- require_test_files: true — requires at least one test_*.py to exist
- Policy is evaluated after anomaly detection, before tree snapshot
- 0% breach rate guarantee — hard block on violations
- Error output names the exact files and rules that failed
- --force still overrides policy (emergency escape hatch)

15 Object Lineage — Dataset & Model Tracking

Module: lineage.py

🧬 Automatic AI Asset Fingerprinting

Reproducibility | Phase 9

Automatically discovers and SHA-256 fingerprints all large AI assets referenced in the project, linking them to the commit history for perfect reproducibility.

- Scans for: .bin, .onnx, .pth, .pt, .h5, .csv, .jsonl, .parquet
- Generates SHA-256 hash + file size for each discovered asset
- Stores fingerprints in .vault/metadata/lineage.json
- Linked to the commit when the asset was first detected
- Allows any researcher to verify they have the exact model/dataset version
- Displayed visually: 🧬 Object Lineage Detected on every commit

16 AI Audit Trail — Transparency Layer

Module: audit.py

📋 Hidden AI Audit Log (.vault/.audit_log)

Transparency | Phase 9

Every AI interaction is recorded in an append-only hidden log inside the repository, giving teams full visibility into what AI decided and why.

- Appends a JSON line on every LLM call
- Records: timestamp, action type, provider/model used
- Records: system prompt sent (first 500 chars)
- Records: user prompt sent (first 500 chars)
- Records: confidence score if available
- Used for compliance, audits, and debugging unexpected AI behavior
- Never truncated — full append-only audit trail

17 Performance Benchmark Tracker

Module: cli.py

⏱ Cycle-Time Monitor — Sub-100ms Commits

Performance | Phase 9

Every commit prints the wall-clock milliseconds taken to snapshot and store every file in the object store, proving performance superiority over legacy VCS.

- Wraps direct_commit() with Python time.time() measurements
- Prints: ⏱ Cycle-Time Benchmark: Vault-AI finalized snapshot in X.XXms
- Benchmarked: ~18-22ms for 100+ file repositories on Apple Silicon
- Eliminates git's index lock, staging area, and delta compression overhead
- Verification: 100 files committed in 0.099 seconds (vs ~0.5-2s for git)

18 Remote Sync & Collaboration

Module: server.py + store.py

☁️ vault sync — Remote Object Push

Team

Pushes all local Vault-AI objects to a remote server instance for team collaboration.

- vault sync --url http://server:8000
- Pushes all objects/ directory items over HTTP
- server.py provides the receiver endpoint
- Default URL: http://127.0.0.1:8000